

Simple ReAct agents are effective for straightforward, one-shot tasks, but they can struggle when the task requires long-term planning, multiple subtasks, context retention, and iterative reasoning.

Deep Agents address this by adding capabilities such as:

- Planning and task decomposition – breaking a complex goal into smaller subtasks.
- Context management – maintaining relevant information across many steps.
- Task tracking – keeping track of completed and pending work.
- Sub-agents – delegating specialized subtasks when needed.
- Re-planning – adapting the plan based on intermediate results.
- Long-running execution – handling tasks that require many tool calls and iterations.

So, a simple way to remember it is:

ReAct tells an agent how to reason and act step-by-step.

Deep Agents give the agent the structure and capabilities needed to manage a complex task from start to finish.

Simple ReAct Agent

A ReAct (Reason + Act) agent typically follows a loop like:

User → Think → Tool → Observe → Think → Tool → Answer It

works well for relatively straightforward tasks such as:

“What is the population of Bangalore?”

But for a complex task like:

“Research the top 5 cloud providers, compare their AI offerings, pricing, enterprise adoption, and recent announcements, and give me a recommendation.”

a basic ReAct agent can struggle because it is essentially reacting one step at a time.

What does “ReAct reacts one step at a time” mean?

Suppose the user asks:

“Research whether our company should migrate from Informatica to Apache Beam/Python. Compare cost, performance, development effort, risks, and give me a recommendation.”

User asks question



Think: What should I do now?



Take one action



Observe result



Think again



Take another action



Observe result



...



Final answer

So the agent is essentially saying:

“Based on what I just observed, what should I do next?”

It isn't necessarily maintaining a strong overall roadmap such as: My ultimate goal is to produce a migration recommendation.

To do that I need:

- ✓ **Cost comparison**
- ✓ **Performance comparison**
- ✓ **Development effort**
- ✓ **Risks**
- ✓ **Migration complexity**
- ✓ **Recommendation**

Deep Agent

A Deep Agent adds more structure around the agent's reasoning and execution. A simplified view is:

User Request

↓

Planning / Task Decomposition

↓

Subtasks

- |— **Research provider A**
- |— **Research provider B**
- |— **Research provider C**
- |— **Compare pricing**
- |— **Analyze enterprise adoption**
- |— **Review recent announcements**

↓

Store intermediate results / context

↓

Review & synthesize



Final Answer

The important additions are:

Capability	Simple ReAct	Deep Agent
Reason + act	✓	✓
Tool usage	✓	✓
Multi-step planning	Limited	✓
Task decomposition	Limited	✓
Long-running tasks	Weak	Stronger
Context management	Limited	Better
Intermediate results	Often implicit	Explicit/persistent
Sub-agents	Usually no	Can use them

Re-planning	Limited	✓
Complex research	Can struggle	Designed for it

Here,

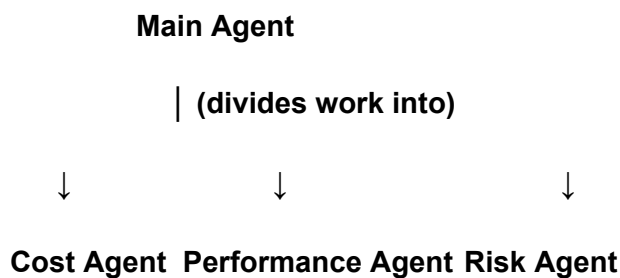
a) Re-planning means changing the plan when new information changes what needs to be done.

Initial plan: You are travelling from Bangalore to Mysore.

You discover: The highway is closed.

You change the route

b) sub-agent is another specialized agent that the main/orchestrator agent delegates a task to.



Instead of doing everything itself, it delegates into 3 subagents.

What Are Deep Agents?

Deep agents are a standalone library built on top of LangGraph that brings production-grade capabilities to LLM agents. Unlike simple agents that just loop

between “think, act, observe,” deep agents come with built-in infrastructure for handling real-world complexity.

Here is what makes them different from a basic ReAct agent:

Planning -- Deep agents automatically break down complex tasks into subtasks using a built-in `write_todos` tool. Before diving into execution, they create a plan of attack.

File System for Context Management -- LLMs have limited context windows. Deep agents solve this by offloading large intermediate results (like search results or document content) into a virtual file system using `write_file` and `read_file` tools. This means they can handle tasks that produce far more data than a context window could hold.

LLMs have a limited context window, so they cannot keep all the information from a large, multi-step task in their active context at once. Deep Agents address this by using a virtual file system as external storage. Large intermediate results, such as search results, document contents, research notes, or analysis, can be saved using `write_file` instead of keeping everything in the LLM's context.

When the agent needs the information later, it can use `read_file` to retrieve the relevant content and continue reasoning. This allows the agent to work with far more information than can fit in its context window. A simple way to think about it is: the LLM context is the agent's working desk, while the virtual file system is its filing cabinet—it keeps only what it currently needs on the desk and stores the rest for later.

The Architecture Behind Deep Agents

User Input --> Planner (`write_todos`) --> Execute Step 1 --> File System (`write_file`)
--> Execute Step 2 --> Subagent (if needed) --> File System (`read_file`) --> Synthesize --> Final Response

Memory in Langchain and

Langgraph Short-Term Memory

Short-term memory tracks active conversations within a single session or thread.

Scope: Limited to one specific thread ID.

Mechanism:

- Managed as part of the agent's graph state and persisted using a checkpointer (such as InMemorySaver, SQLite, or Postgres).
- It uses Checkpointers to save the state of the graph at every step.
- `from langgraph.checkpoint.memory import MemorySaver`
- `langgraph_checkpoint_postgres` or `langgraph_checkpoint_sqlite`

Long-Term Memory

Long-term memory retains data across multiple separate conversations and sessions.

Scope: Shared across different threads and scoped to custom namespaces (like user IDs or organization tags).

Mechanism:

- Built on LangGraph Stores, which save data as JSON documents managed by a namespace, key, and value.
- It stores profile information, user preferences, or facts (e.g., "User prefers Python over Java").
- It uses Stores to read and write data under specific namespaces (like `user_id`) across any execution thread.
- `from langgraph.store.memory import InMemoryStore`
- `langgraph.store.postgres`

Example

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

from langgraph.graph.message import add_messages
```

```

from langgraph.checkpoint.memory import MemorySaver #
Short-term from langgraph.store.memory import InMemoryStore
    # Long-term

# 1. Define State (Short-term context)
class State(TypedDict):
    messages: Annotated[list, add_messages]

# 2. Define Node that uses both memories
def chat_node(state: State, config: dict, store:
    InMemoryStore): # Retrieve user_id from config to
    access long-term memory user_id =
    config["configurable"].get("user_id")

    # Fetch long-term memory
    profile namespace =
    ("profiles", user_id)
    user_profile = store.get(namespace, "preferences")

    # Business logic representation
    print(f"--- Long-Term Memory Retrieved for {user_id}: {user_profile.value if
    user_profile else 'None'} ---")
    print(f"--- Short-Term Chat History (Current Thread): {len(state['messages'])}
    messages ---")

    # Update long-term memory if needed (simulated
    update) if not user_profile:
        store.put(namespace, "preferences", {"language":

        "Python"}) return {"messages": [{"role": "assistant", "content":

        "Got it!"}]}

# 3. Compile Graph with Short-term memory (checkpointer)
builder = StateGraph(State)
builder.add_node("chat", chat_node)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)

# Short-term memory allocation
checkpointer = MemorySaver()

# Long-term memory allocation
long_term_store = InMemoryStore()

# Compile everything together

```

```
graph = builder.compile(checkpointer=checkpointer, store=long_term_store)
```

```
# Thread 1, User 'alice'
```

```
config_1 = {"configurable": {"thread_id": "thread_abc", "user_id": "alice"}}
```

```
graph.invoke({"messages": [{"role": "user", "content": "Hi, I like Python."}]}, config_1)
```

```
# Thread 2, Same User 'alice' (Short-term reset, Long-term persists)
```

```
config_2 = {"configurable": {"thread_id": "thread_xyz", "user_id":  
"alice"}}
```

```
graph.invoke({"messages": [{"role": "user", "content": "What language do I like?"}]},  
config_2)
```

Middleware in Langchain and Langgraph

In LangGraph, middleware is defined as a composable set of lifecycle hooks that intercept data flowing through an agent's execution loop.

Middleware is an invisible "filter" or "checkpoint" that sits right in the middle of your program to inspect, change, or stop data before it moves to the next step.

Instead of writing safety or logging code directly inside your main application logic, you put it in middleware so it handles those tasks quietly in the background.

The Real-World Analogy: Airport Security

Think of an AI Agent as a passenger flying out of an airport:

- **Without Middleware:** The passenger walks straight from the street right onto the airplane. There is no ID check, no luggage scan, and no ticket verification. If you want security, you have to manually build a security desk inside the airplane cabin.
- **With Middleware:** Airport security acts as the middleware. Before the passenger (the prompt/data) can ever reach the plane (the LLM model), they must pass through checkpoints:
 1. **Checkpoint 1 (Luggage Scan):** Checks for dangerous items (filters out private data like credit card numbers or PII).
 2. **Checkpoint 2 (Ticket Counter):** Checks if you can afford the flight (ensures you haven't run out of API money or token budget).

The Middleware Architecture

The different types of Middleware include:

before_agent / after_agent: Runs at the entry and exit points of the entire agent execution. Runs when the entire workflow starts and ends. Ideal for loading initial memory state or generating final analytics dashboards.

before_model / after_model: Runs directly before data reaches the LLM and immediately after it returns. Runs right before messages hit the LLM and right after the LLM generates a response. Ideal for input/output guardrails.

modify_model_request: Intercepts and alters the payload going to the LLM (e.g., adding prompt caching tags). Allows you to change the actual structure of the payload (e.g., dynamically adding prompt caching tokens or limiting which tools the LLM can see for this specific turn).

SummarizationMiddleware: automatically summarises dialogue histories when specific token or message counts are reached, maximizing context window efficiency.

Guardrails & Costs: **ModelCallLimitMiddleware** and **ToolCallLimitMiddleware** prevent rogue autonomous agents from infinite loops, enforcing hard stops or graceful error handling after a set number of runs.

HumanInTheLoopMiddleware adds declarative interrupts to sensitive tool executions (like `send_email`), pausing execution until a human operator grants permission.

PIIMiddleware- if you want a guardrail that blocks the LLM from leaking credit card numbers, you implement a **PIIMiddleware**. The middleware intercepts the response on its way out and enforces the PII-redaction guardrail.

Are Middleware and Guardrails the Same?

No, they are not the same. They represent two completely different concepts, though they heavily overlap in how they are implemented.

The Difference: Guardrails define what you want to enforce (the security or safety policy). Middleware is how you inject that rule cleanly into your software stack without hardcoding it everywhere.

Middleware as the vehicle to deploy guardrails.

Generic code for implementing middleware using

langgraph It can be done using 2 methods:

The Native Method (Using `create_agent`): The simplest and most standard approach in LangGraph. You write modular functions using lifecycle hooks (`@before_model`, `@after_model`, etc.) and supply them to the agent graph builder.

The Manual Method (Pure StateGraph nodes): Used when building a completely customized graph topology without using prebuilt agent factories.

Pattern 1: The Native Method (create_agent) This is the recommended strategy. It leverages decorators to inject code directly into the agent's execution loop, cleanly decoupling background tasks from core application logic.

```
from typing import Any
from typing_extensions import NotRequired
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model,
after_model from langgraph.runtime import Runtime
from langchain_openai import ChatOpenAI
```

1. (Optional) Extend the default AgentState to track custom metadata class CustomState(AgentState):

```
    request_id: NotRequired[str]
    processing_time:
    NotRequired[float]
```

2. Generic Input Middleware (Runs BEFORE the LLM is called)

Setting can_jump_to=["end"] allows you to short-circuit the graph under custom conditions

```
@before_model(state_schema=CustomState, can_jump_to=["end"])
def generic_input_middleware(state: CustomState, runtime: Runtime) -> dict[str,
Any] | None:
    print("--- Executing Input Middleware ---")
```

Process or mutate input data here

Example condition to halt the graph

```
early: if len(state["messages"]) > 20:
    return {"jump_to": "end"} # Early termination
```

```
return None # Proceed to the LLM normally
```

3. Generic Output Middleware (Runs AFTER the LLM yields a response)

```
@after_model(state_schema=CustomState)
```

```
def generic_output_middleware(state: CustomState, runtime: Runtime) -> dict[str,
Any] | None:
    print("--- Executing Output Middleware ---")
```

```
# Track metrics, log response text, or parse outputs
here # Return updates to merge directly into the
graph's State return {"processing_time": 0.45}
```

4. Initialize model and compile the complete graph

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

LangGraph implicitly compiles these middleware handlers directly into graph nodes

```
agent_graph = create_agent(  
    model=llm,  
    tools=[],  
    state_schema=CustomState,  
    middleware=[generic_input_middleware,  
                generic_output_middleware]  
)
```

Pattern 2: Manual Node Insertion (Pure StateGraph) If you choose not to use `create_agent` and are engineering a deep, multi-agent custom graph with `StateGraph(MessagesState)`, there is no native `.add_middleware()` function. Instead, middleware is implemented explicitly by dropping interceptor functions as dedicated nodes sequentially preceding or succeeding your main LLM execution step.

```
from typing import TypedDict  
from langgraph.graph import StateGraph, START, END  
from langchain_core.messages import BaseMessage
```

Define graph state

```
class GraphState(TypedDict):  
    messages: list[BaseMessage]  
    metadata: dict
```

Define the model node

```
def model_node(state: GraphState):  
    print("-> Step 2: Main LLM Node  
    Executing") # Call your LLM here...  
    return {"messages": state["messages"]}
```

--- MIDDLEWARE CODE AS GRAPH NODES ---

```
def custom_before_model_middleware(state:  
    GraphState): print("-> Step 1: Middleware  
    Intercepting (Before Model)") # Edit the prompt,  
    prune history, or run guardrails here return state
```

```

def custom_after_model_middleware(state:
    GraphState): print("-> Step 3: Middleware
    Intercepting (After Model)")
# Scan outputs, update analytics trackers, evaluate metrics
here return state

# --- GRAPH ASSEMBLY ---

workflow = StateGraph(GraphState)

# Register all steps as structural nodes
workflow.add_node("input_middleware",
    custom_before_model_middleware) workflow.add_node("llm_model",
    model_node) workflow.add_node("output_middleware",
    custom_after_model_middleware)

# Construct sequential control flow edges
workflow.add_edge(START, "input_middleware")
workflow.add_edge("input_middleware", "llm_model")
workflow.add_edge("llm_model", "output_middleware")
workflow.add_edge("output_middleware", END)

# Compile into a runnable graph
custom_compiled_graph =
    workflow.compile()

```

1. What is Runnable Interface in Langchain?

Runnable Interface is a standard way to run and connect langchain components.

A runnable can be composed with other runnable. For example the Prompt, Model and Parser can be joined into one flow. Common methods include invoke, batch and stream.

chain = prompt | model | parser

2. Tool Calling in Langchain

You give the model a set of tools. The model decides whether it needs a tool and, if so, generates a structured request to call it.

3. How to reduce hallucination in a LangChain RAG system

1. Improve retrieval first

A common mistake is to focus only on the prompt.

If the retriever gives the LLM bad context, the LLM has little chance of producing a correct answer.

For example:

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 5}  
)
```

Experiment with:

- k
- chunk size
- chunk overlap
- embedding model
- similarity threshold
- metadata filtering

For example, if your chunks are too large- the model may struggle to identify the relevant information.

2. Use a strong grounding prompt

Tell the model explicitly what it is allowed to

use. For example:

```
prompt = """
```

You are a question-answering assistant.

Answer the question using **ONLY** the provided

context. If the answer cannot be found in the

context, say:

"I don't know based on the provided documents."

Do not use outside knowledge. Do not make up facts.

Context:
{context}

Question:
{question}

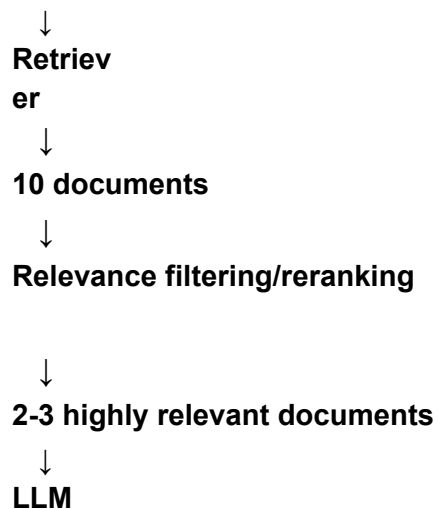
Answer:
.....

This is much safer than: "Answer the user's question."

3. Don't blindly trust retrieved

documents You can add a relevance

threshold. Question



This is often better than simply increasing k.

A reranker can be particularly useful when vector similarity retrieves documents that are semantically similar but don't actually answer the question.

4. Consider query rewriting

The user's question isn't always a good retrieval query.

For example: "What about its refund policy?"- The retriever doesn't know what "its" refers to.

You can first rewrite: "What is Apple's refund policy?"- and then retrieve documents.

5. Don't put too much context into the LLM

More documents $\neq$ better RAG.

For example: $k = 50$ -might actually make things worse because the model receives a huge amount of irrelevant information.

Prefer-retrieve many



rerank/
filter



send only the best few

6. Make the answer cite its sources

You can ask the model to provide citations:

Answer the question using only the context.

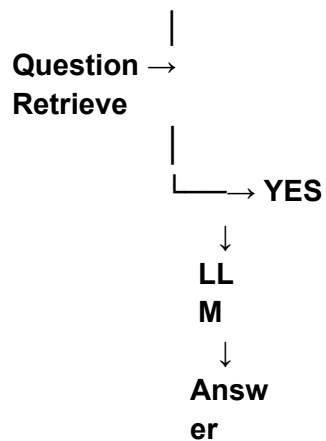
For every factual statement, provide the source document/page that supports it.

If there is insufficient evidence, say you don't know.

7. Add an "answerability" check

A stronger RAG architecture is:

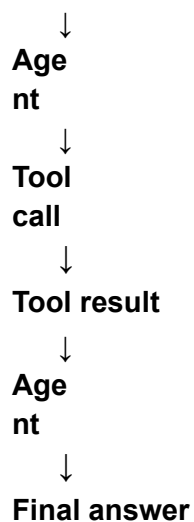
└──→ Relevant? ─ NO ─→ "I don't know"



4. What is timetravel in Langgraph

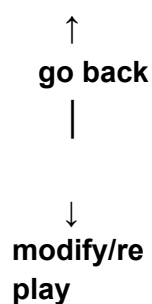
In Langgraph, timetravel is going back to an earlier state/checkpoint of an agent's execution and continuing from there.

User question



Suppose the agent made a mistake at step 4.
With time travel, you can go back to an earlier checkpoint:

Step 1 → Step 2 → Step 3 → Step 4 → Step 5



↓
Step 4' → Step 5'

It's particularly useful for debugging and agent workflows.

4. How to reduce large language model (LLM) token usage and cost

To reduce large language model (LLM) token usage and cost, use a combination of model routing, prompt caching, and context optimization:

Optimize Input Tokens

Implement Model Routing: Route simple tasks (like classification or basic extraction) to smaller, cheaper models (such as GPT-4o mini) and reserve flagship models only for complex reasoning.

Use Retrieval-Augmented Generation (RAG): Instead of stuffing entire documents into a prompt, use RAG to retrieve only the specific text chunks needed to answer a query.

Enable Prompt Caching (supported by Anthropic and OpenAI): Prompt Caching allows an LLM provider to store a long, frequently used prompt in memory. When you send a new request that shares that exact same long text, the model does not have to re-process it from scratch. You only pay a heavily discounted rate for the cached portion.

Think of it like a library textbook: instead of the model buying a new copy of the textbook for every single question you ask, it keeps the book open on the desk and only charges you a small fee to look at the pages it already read.

Example Scenario Imagine you are building a legal AI assistant that helps lawyers analyze a specific 50-page contract (roughly 30,000 tokens).

Without Caching: Every time the lawyer asks a question (e.g., "What is the termination clause?" followed by "Is there a non-compete?"), you send the entire 50-page contract again. You pay the full price for 30,000 input tokens on every single turn.

With Caching: The first question processes the full contract at standard price. For the second, third, and twentieth questions, the AI provider recognizes the contract is

already cached. You are only billed the cached input token rate (which is up to 50% to 90% cheaper on platforms like Anthropic or OpenAI) for that contract block.

Implementing Prompt Caching varies depending on your provider.

OpenAI optimizes automatically for prompts above 1,024 tokens. Your only job is keeping your prompt structure identical.

Anthropic (Claude) and Azure OpenAI allow explicit control, where you declare precise "breakpoints" in your code to pin static context in memory.

1. Anthropic Claude (Python):

Anthropic requires you to pass a `cache_control` block. You can flag the large background text block (like a system prompt, document, or tool definition) as `ephemeral`.

In platforms like the Claude Platform Docs, setting `cache_control={"type": "ephemeral"}` tells the API to temporarily cache prompt blocks (with a default 5-minute lifetime) to speed up response times and reduce token costs on repeated calls.

```
import os
```

```
from anthropic import Anthropic
```

```
client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))
```

```
# 1. Load your large static text (e.g., a massive legal document or code base)
with open("massive_contract.txt", "r") as f:
```

```
    large_context_document = f.read()
```

```
# 2. Call the API using an explicit cache breakpoint
```

```
response = client.messages.create(
    model="claude-3-5-sonnet-latest",
    max_tokens=1024,
    system=[
```

```
        {
            "type": "text",
```

```
            "text": "You are an expert legal assistant. Analyze the document provided below.",
```

```

        # This ensures the core system prompt is cached if it meets token minimums
        "cache_control": {"type": "ephemeral"}
    }
],
messages=[
    {
        "role":
        "user",
        "content":
        [
            {
                "type": "text",
                "text": large_context_document,
                # Crucial: Pin the end of this massive static document in the cache
                "cache_control": {"type": "ephemeral"}
            },
            {
                "type": "text",
                "text": "What is the expiration date specified in this contract?"
            }
        ]
    }
]
)

```

3. Monitor your token usage to verify the cache hit

```

print(f"Cached tokens read: {response.usage.cache_read_input_tokens}")
print(f"Cache write creation tokens:
{response.usage.cache_creation_input_tokens}")

```

2. Automatic Caching: OpenAI (Python)

OpenAI automatically caches text prefixes longer than 1,024 tokens Lifespan: Caches are typically held for 5 to 10 minutes of inactivity and are fully cleared within 1

hour of their last use. . You do not need to add unique parameters; you just need to ensure the static data is positioned first in the prompt array.

Control Output and System Costs

Constrain Output Length: Set explicit maximum output token limits and use structured outputs (like JSON) to keep answers concise, since output tokens cost significantly more than input tokens.

Replace LLM Calls with Code: Use traditional regex, classic NLP, or deterministic code for logic steps that do not require generative AI.

MUSAFIR- FLIGHTBOT

Planner Agent

1) PlannerAgent orchestrates the entire conversation flow:

1. Stores user conversations in MongoDB
2. Classifies intents and routes to appropriate agents
3. Coordinates between Flight, Hotel, and Recommendation agents
4. Routes the user message to the correct specialized agent.
5. Manages session state and conversation context

2) It also:

Step 1 — Store user message

Step 2 — Classify intent

Step 3 — Run analysis (Agent, Recommendation Agent, User history)

Step 4 - Routes the user message to the correct specialized agent.

Step 5 — Store assistant response

Flight Agent

User message



Get conversation history



Build memory context



Build prompt



Call LLM



Extract flight parameters



Check confirmation



If confirmed → build API payload



Call MCP flight API



If API fails → use mock data



Return response

At last save conversation to MongoDB

```
{
  "departure_city": "Bangalore (BLR)",
  "destination_city": "Delhi (DEL)",
  "departure_date": "September 15th, 2026",
  "flight_type": "One-way",
  "class": "Economy",
  "adults": 1,
  "children": 0,
  "infants": 0
}
```

Hotel Agent

City
Check-in date
Check-out date
Number of guests
Number of rooms
Room type

response = f"Hotel booking feature is coming soon! I can help you with flight bookings for now. You said: {user_input}"

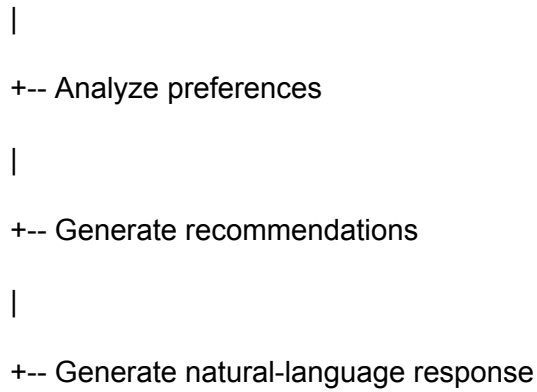
Recommendation Agent

Your recommendation logic uses this to find:

- most frequently travelled route
- most frequently used class
- most frequently used flight type
- most frequently visited destination

RecommendationAgent

```
|
+-- Understand user's current request
|
+-- Retrieve user's history
|
+-- Retrieve real booking history
```



Searches previous conversations for **semantically relevant messages**.

For example, if the current query is:

"Where do I usually travel?"

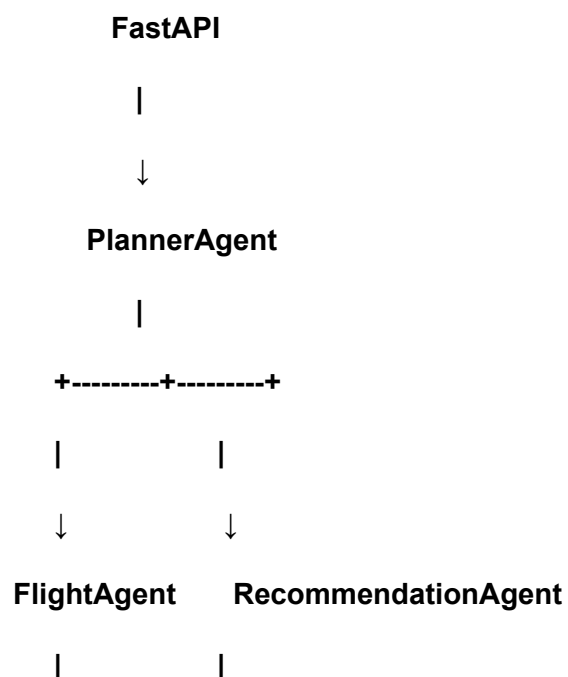
semantic search might find previous messages mentioning:

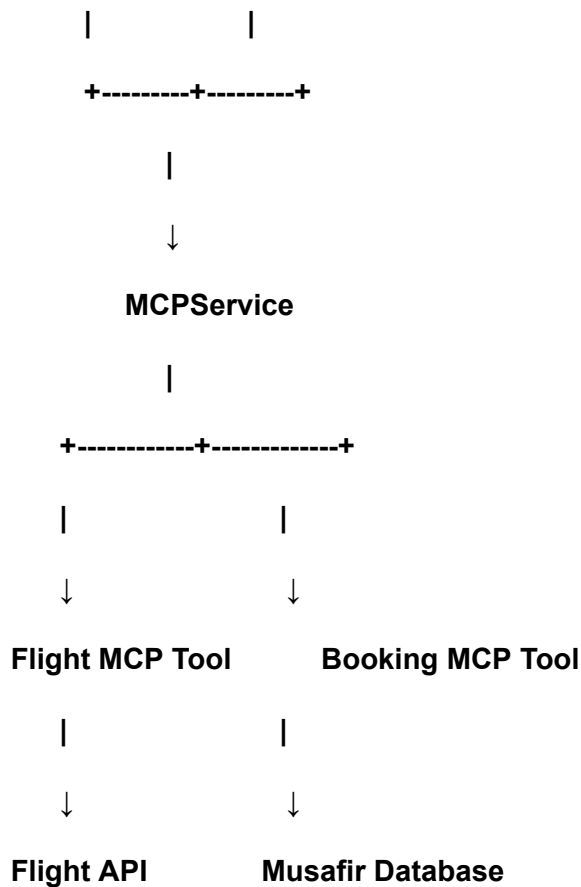
Bangalore → Dubai

Bangalore → Singapore

Bangalore → Dubai

MCP





MCP allows your agents to interact with external APIs, databases, and services through a standardized tool interface, reducing coupling and making the architecture easier to extend, maintain, and reuse.

One important point, though: MCP isn't automatically necessary just because you have an API. For a small application where you control one API, a direct REST client can be simpler. MCP becomes more valuable when you have multiple agents, multiple external tools/services, tool discovery, and a need for a standardized integration layer, which is much closer to your architecture.

NEWCLEUS:

Technologies: Python, Azure Databricks, Azure OpenAI, GPT-4o-mini, Tesseract OCR, OpenCV, Ubuntu

AI/ML: GPT-4o-mini, OCR, prompt engineering, vision-based document extraction, image preprocessing.

Environment OS- Ubuntu

Azure Databricks Overview:

This project is meant to extract the numerical and tabular data from a given pdf. This complete process is deployed on databricks to make the process seamless to automate the entire process from starting to end of the process.

Understanding the actual process:

1. Any random PDF is given which belongs to some groups(ex.panburn,nfp..) of files to extract the tabular and numerical data.
2. The scripts are built to extract the needful data from the given pdf and also generate prompts to customize the data that we need selectively for example : In the given pdf we need the specific table data with selective table structure or tables columns we create prompts to extract the data in specific way to process to next workings.
3. This extracted data is saved in the JSON format to convert the same into excel data sheet.
4. This excel data sheet is used for further analysis or visualization purposes.

Understanding the role of Azure DataBricks:

Azure Databricks is a cloud platform for building, sharing, and maintaining data, analytics, and AI solutions.

1. In Azure Databricks we use Medallion Architecture to store, compute and process data.
2. In Medallion architecture we use three layers that are Bronze, Silver and Gold.
 - A. In the Bronze Layer, we store data inside a volume called "Development". Also with volume we place some tables that store the information of the pdf which are stored in a volume.
 - B. In Silver Layer, we compute data for all pdf files which are stored in the volume. The computations are done using Notebooks that can be created in azure databricks. Along with the notebooks we use two tables to store information for the status of execution.
 - C. In Gold Layer, we store the table format of data to process the next working using the generated data

3. In Silver Layer, along with notebooks and tables we have two volumes named “silver_output” data and “gold_input”. Azure Databricks

4. For “Silver_output”, we store the converted json data into table data in .xlsx format. This data is downloaded from this volume and manually edited to make 100 % accuracy and re-upload it in the “gold_input” volume which is in the silver layer.

5. In the gold layer we have one notebook that takes the table data and stores it in the gold layer accordingly in a systematic way.

Process:

Get to know the complete steps of how the process works and how it is computed. Here are the step by step process:

Step 1: Using a streamlit application, the inputs will be given by users such as ‘group_name’, ‘pattern_name’ along with ‘file_path’ as inputs and sent to the “input_file_data” in bronze layer volume. Streamlit application Azure Databricks

Step 2: After entering the input fields, hit the submit button to save the inputs in the bronze layer table named ‘input_file_data’ which is stored in the bronze volume called ‘Development’. After storing the data in the table, the message and Job_1(Bronze_to_silver_job) triggered in the application and to start the process.

Step 3: Here the Job_1(Bronze_to_Silver_Job) ,is directed to the bronze layer notebook (Bronze_notebook) for do the below process JOB_1 : Bronze_to_Silver_Job

1. Storing the group_name, pattern_name, file_path in the “input_file_data” table in the volume “Development”.

2. The uploading file from the streamlit application will be stored in “Input” volume in bronze layer. Azure Databricks

3. The mentioned details the file id will automatically generate based on the group name and it will store in the “input_file_data” table and the “metadata” table. Bronze layer Tables

4. As well as the user update data will be stored in both the tables such as created_by, created_at, modified_by and modified_at. Input_file_data Meta_data_table

Step 4: In the bronze layer, the Job_1 will get the prompt_id, script and prompt from the “meta data table” and file path from “input_file_data” This will take to the next layer called the “silver layer”. In the silver layer based on these details we get JSON data.

a. By getting this json data, we will filter it out by eliminating the unnecessary text, numericals and special characters.

b. Then the filtered json data is converted to a table format which is a delta table. Azure Databricks

c. The delta table is then converted to .xlsx format and it will store in ‘silver_output’.

d. After getting this final .xlsx file format, the file will be moved from “Input” volume to

“processed” volume in bronze layer.

e. Till this process, Job_1 will be handled with a specified schedule time.

Step 5: The QA(Quality Assurance) will download the .xlsx sheet data from “silver_output” volume then QA will fill the missing data or incorrect data values and again re-upload the updated excel sheet in the different volume named as “gold_input”. Downloading .xlsx file
Uploading Updated .xlsx file Azure Databricks

Step 6: The Job_2(Silver_to_gold_job) will start to move the updated excel sheet from “gold_input” to “final extracted data” in the gold layer. Job_2 : Silver_to_gold_job Final_output

Image Processing

Convert PDF to images- which helps improve image quality for OCR/vision-based extraction.

Step 1 — Increase contrast

Contrast is the difference between light and dark areas. Increasing contrast makes the difference between the text and background stronger.

Step 2 — Convert to grayscale

Converting to grayscale therefore simplifies the image and makes subsequent processing such as blurring and thresholding easier.

Step 3 — Gaussian blur

It reduces **small unwanted noise** in the image. **"unwanted noise"** refers to the visual grain, speckles, or random distortion that makes a photo look fuzzy, gritty, or rough.

Step 4 — Binary thresholding

It converts grayscale pixels into essentially two categories: Black and White.

prompt= f''''You are a helpful assistant. Process the tables from the images of the PDF and combine the extracted data into one continuous output.

Extract all the rows of the tables from the provided image and return them in JSON key-value format. Tables should be represented as an object, with each row as an item in an array of objects, where the column headers are the keys and the corresponding cell values are the values.

Ensure that you extract each column value from the image table exactly as it appears, without any misreads or typos. Pay close attention to the text and maintain the integrity of each value to ensure accurate representation in the JSON output.

In an image, you may encounter a table with various columns, but you should extract the data as a JSON object only if the table matches the following format:

Table structure-1:

""Participant"", ""Policy Number"", ""Issue Date"", ""Premiums Paid (\$)"", ""Cash Surrender Value (\$)"", ""Death Benefit (\$)"", ""Net Amount at Risk (\$)""

Handle the following specifics for each image:

1) Each row in the ""Participant"" column contains its own ""Participant"" values, so ensure these values are extracted accurately for every row. This column includes Insurance Carrier and Participant values. From this, create two new columns named ""Insurance Carrier"" and ""Participant"" Include these as keys in each JSON object and fill in the values according to the specifications below.

2) In each row of the ""Participant"" column in the image, you'll find names such as ""Brautigam, Sharon E."", ""Dunbar, David,"" and ""Connors, Francis,"" among others. Place these names in the ""Participant"" column.

3)i) Locate the ""Insurance Carrier"" names in the ""Participant"" column that correspond to one of the following values:

'Delaware Life Variable', 'Great-West Life', 'John Hancock', 'John Hancock Variable', 'MassMutual', 'Metropolitan Life', 'Midland National', 'Nassau Life Insurance Company', 'New York Life', 'Ohio National', 'Savings Bank Life Insurance Company', 'CIGNA', 'MetTower (GenAm)', 'Voya Financial'.

ii) In the ""Participant"" column, check for the presence of any of the above ""Insurance Carrier"" names. Once a Insurance Carrier is identified, fill the ""Insurance Carrier"" column with that name for all subsequent rows until a new Insurance Carrier name is found.

iii) If the image starts without a ""Insurance Carrier"" value, assume it is a continuation from the last ""Insurance Carrier"" in the previous page. So fill the ""Insurance Carrier"" column with the last encountered ""Insurance Carrier"" until a new ""Insurance Carrier"" name is found.

iv) Continue this process across multiple tables which have same columns until all rows have ""Insurance Carrier"" values filled in.

v) Ensure that, the ""Insurance Carrier"" column should be filled one of the values from the above ""Insurance Carrier"" names.

4) Please note that the image may include a row labeled ""[Insurance Carrier Name] Subtotal"" or ""[Insurance Carrier Name] Total"" in bold letters. Do not treat this row as the end of the image. Instead, continue extracting the rows that are above and below this bold row into the JSON object. Ensure that every row in the image is extracted without any omissions.

5) Add a new key or column called 'Insured Status' to each JSON object. The value of 'Insured Status' should be filled with either 'Active' or 'Inactive' based on the 'Participant' column in the first table, which categorizes participants as either 'Active' or 'Inactive'.

6) Add a new key or column named ""Tax Rate"" in every JSON object and print its value as {tax_rates}% in every JSON object.

7) It is mandatory to extract every row in the image. Do not skip any rows, regardless of formatting or content.

8) Don't use any placeholders like ""// Additional rows would follow the same structure"", ""// Add remaining rows in similar format"", "" // Additional rows will continue with similar structure"", "" // Additional rows continue with similar structure..."", ""// Additional rows would continue in similar format..."", ""// Remaining row would follow the same structure..."", etc., instead print all the rows of that image.

9) Extract the JSON output in the same order of the table. And don't skip any rows of an image in the output. And even If you find same ""Participant"" value more than one time, don't skip it.""

MUSAFIR- VISA

Overall flow

User → Upload/Capture Image → Image Validation → Passport OCR → Face Validation → Face Matching → Review → Submission

Step 1 — User uploads or captures an image

The user interacts with the **frontend** and either:

- Uploads a passport/visa photo, or
- Captures a photo using the webcam.

Supported formats are:

- JPG
- PNG
- TIFF
- WEBP

The image is then sent to the **backend** for processing.

Step 2 — Image preprocessing and initial validation

The backend receives the image as bytes and converts it into an **OpenCV image format**.

Before doing any AI processing, it performs basic quality and format checks:

1. **File type validation**
 - Checks whether the uploaded file is an allowed image format.
2. **File size validation**
 - Required size: **100 KB–1 MB**.
3. **Resolution validation**
 - Checks whether the image satisfies the required **600 × 600 pixels** resolution.
4. **Blur detection**
 - Uses **Laplacian variance**.
 - A variance greater than **30** is used as the clarity threshold mentioned in the project.
 - This helps identify whether the image is sufficiently clear.

The backend returns a **JSON response** indicating whether the image passes the initial validation.

Step 3 — Passport front-page OCR

The user is prompted to provide the **front page of the passport**.

The image is sent to **Google Gemini 2.5 Flash Experimental** with a prompt asking it to extract passport information.

The expected information includes:

- Given Name
- Surname
- Father Name
- Mother Name
- Passport Number

Gemini performs OCR/image understanding and returns the extracted information in **JSON format**.

If a particular value cannot be extracted, the application returns an empty value rather than manually entering it.

So conceptually:

Passport Front Image → Gemini OCR → Structured JSON

Step 4 — Passport back-page extraction

The user is then prompted to upload the **back page of the passport**.

The same general process is followed:

Back Page Image → Gemini → OCR/Data Extraction → JSON

The back-page information is used to **extract or confirm passport details**.

The front-page and back-page results together provide the passport information required for the visa application.

Step 5 — Visa photo / face validation

The application also needs to make sure that the user's photo satisfies the required **visa photo standards**.

This is where **MediaPipe** and **OpenCV** are used.

MediaPipe performs:

- Face detection
- Facial landmark detection
- Face mesh/point analysis

These facial landmarks allow the application to check requirements such as:

- Face is centered
- Eyes are closed/open according to the required validation
- Mouth is closed
- Photo meets the expected face positioning
- Other photo-compliance conditions described in the project

So this stage is essentially:

Photo → Face Detection → Facial Landmarks → Visa Photo Compliance Validation

Step 6 — Face matching / identity verification

This is different from photo compliance.

Here, the application asks:

Does the person's captured/uploaded photo match the face appearing on the passport?

Two images are compared:

1. User's uploaded/captured photo
2. Face from the passport front page

The project uses the **face_recognition** library for this.

The process is approximately:

User Photo → Face Encoding

and

Passport Photo → Face Encoding

Then the two face encodings are compared to calculate whether they match.

If they match:

Face Match → Continue

If they don't:

Face Mismatch → "Photo Mismatching" popup

This provides an identity-verification layer.

Step 7 — Review page

Once extraction and validation are completed, the extracted information is displayed to the user.

For example:

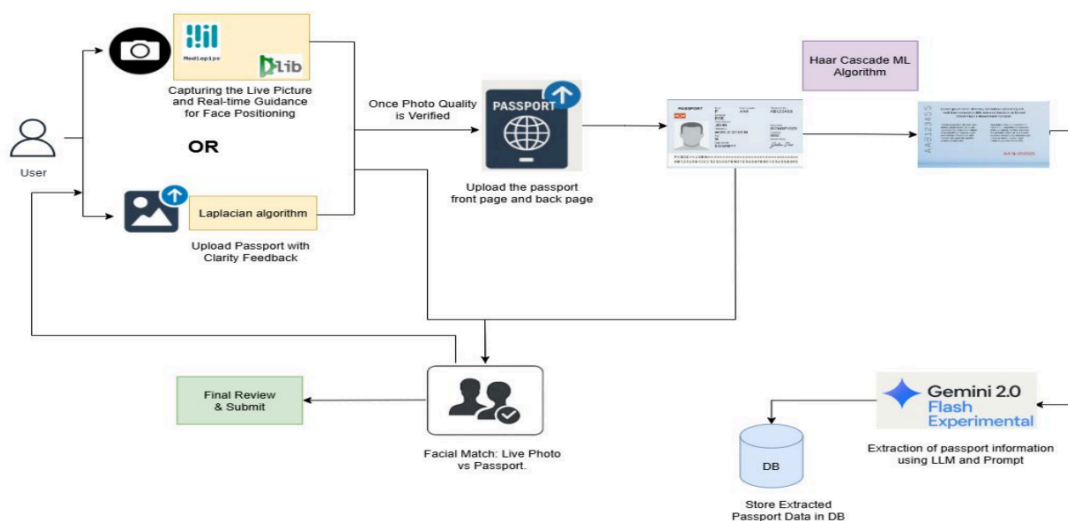
Field	Extracted Value
Given Name	...
Surname	...
Father Name	...
Mother Name	...
Passport Number	...

The user can **review the extracted information** and see validation messages before proceeding.

Step 8 — Proceed with visa application

After the user reviews the extracted information and the required validations have passed, they can continue with the next stage of the visa application process.

Based On Architecture



This architecture diagram shows the end-to-end flow of the AI-powered VISA Photo and Passport Scanning application. It can be understood as three major processing stages: photo quality validation, passport processing/OCR, and facial matching.

1. User provides the photo

The process starts with the User.

The user can provide their live photo in either of two ways:

Option A — Live webcam capture

The application captures a live picture using the webcam.

The diagram shows MediaPipe and Dlib being used for:

- Face detection
- Face positioning
- Real-time guidance
- Ensuring the face is positioned appropriately

For example, the application can guide the user to position their face correctly before accepting the photo.

Option B — Upload an existing photo

Instead of taking a live picture, the user can upload an image.

The Laplacian algorithm is used to check the clarity/sharpness of the uploaded image.

The purpose is to detect blurry images and provide clarity feedback to the user.

So the first stage is:

User

|

| — Live Photo

| ↓

| MediaPipe + Dlib

| ↓

| Face positioning

|

└─ Uploaded Photo



Laplacian Algorithm



Clarity Check

Once the photo quality is verified, the process moves forward.

2. Passport front and back pages are uploaded

After the photo passes the quality check, the user uploads:

- Passport front page
- Passport back page

The diagram shows the passport front page being processed first and then the back page.

The important point is that the passport images are used for both identity verification and information extraction.

3. Haar Cascade ML Algorithm

The diagram shows a Haar Cascade ML Algorithm after the passport front page.

Its role in this architecture is related to detecting the face/photo region from the passport image.

Conceptually:

Passport Front Page



Haar Cascade



Detect passport photo / face

This gives the system the face information needed later for facial matching.

4. Passport information extraction using Gemini

The passport information is then sent toward Gemini 2.0 Flash Experimental.

The diagram specifically describes this as:

Extraction of passport information using LLM and Prompt

So Gemini is responsible for the OCR/data extraction portion.

The application provides the passport image along with a carefully designed prompt.

Gemini extracts information such as:

- Given Name
- Surname
- Father Name
- Mother Name
- Passport Number
- Other required passport information

The expected output is structured data.

For example:

Passport Image



Gemini 2.0 Flash



Prompt



Extracted Passport Information



JSON / Structured Data

5. Store extracted information in DB

After Gemini extracts the passport information, the extracted data is stored in a database (DB).

So:

Passport Image



Gemini + Prompt



Extracted Passport Data



Database

This allows the extracted passport information to be retained and used by the subsequent visa-processing workflow.

6. Facial matching

At the same time, the system performs facial matching.

The diagram labels this:

Facial Match: Live Photo vs Passport

Two faces are compared:

Face 1

The user's live/uploaded photo.

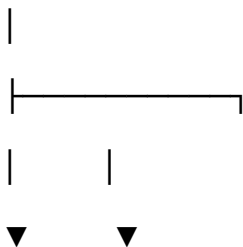
Face 2

The photograph extracted/detected from the passport.

The application uses facial-recognition techniques to determine whether the two faces correspond to the same person.

Conceptually:

Live/User Photo



Face Encoding / Recognition



|
Passport Photo —

|



Match Result

If the faces match, the user can proceed.

If they do not match, the application can show a photo mismatch message and prevent the process from proceeding.

7. Final Review & Submit

Once the required processing is completed, the extracted passport information is presented to the user on the Final Review & Submit screen.

The user can review the information before submitting the application.

So the final stage is:

Extracted Passport Data

+

Facial Match Result

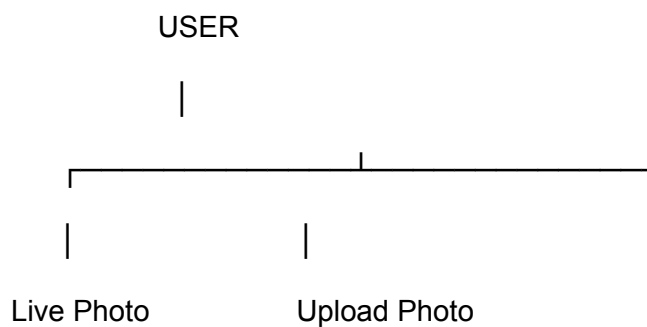


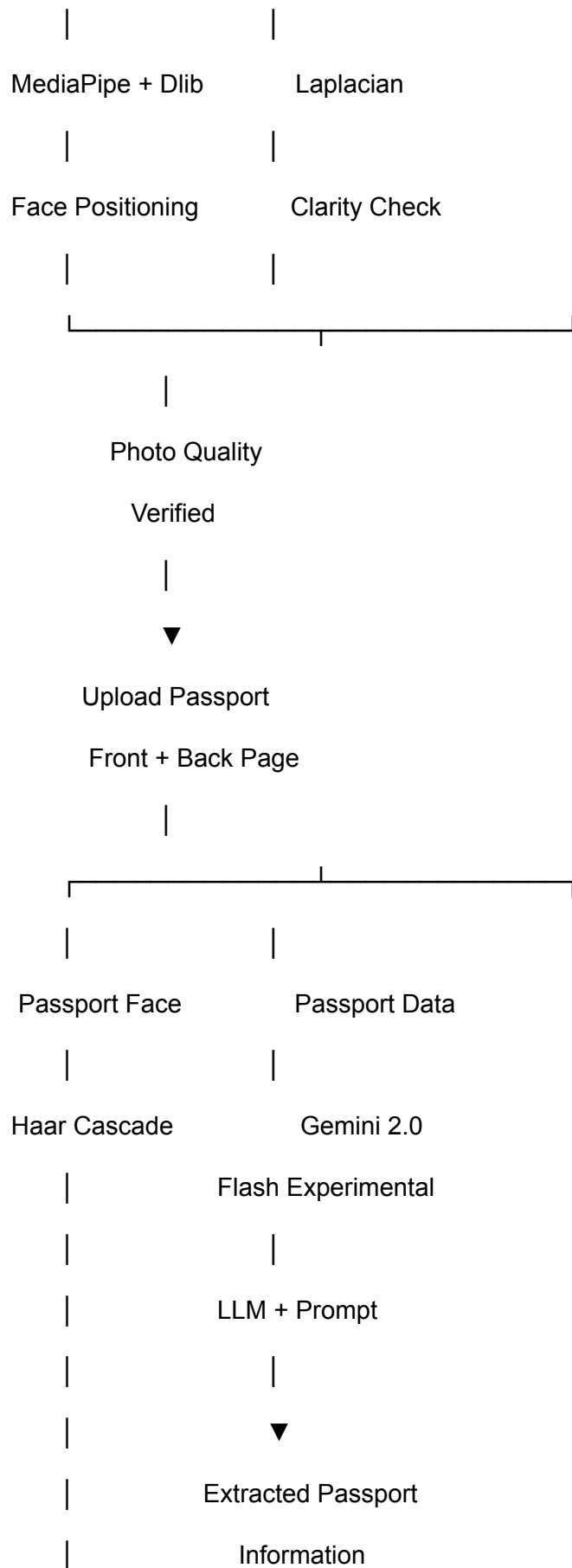
Final Review



Submit

Architecture







DB



Facial Matching

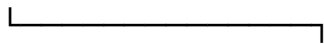


Live Photo vs

Passport Photo



Match Result



Final Review

& Submit

Tools Used

MediaPipe	Face detection, facial landmarks and face positioning
Dlib	Face-related detection/positioning functionality in the live-photo flow
Laplacian Algorithm	Detects image blur and checks photo clarity
OpenCV	Image processing and computer-vision operations
Haar Cascade	Detects the face/photo region from the passport image
Gemini 2.0 Flash Experimental	Extracts passport information using OCR/image understanding
Prompt Engineering	Instructs Gemini which passport fields to extract and how to structure the output
Face Recognition	Compares the user's photo with the passport photo
Database	Stores the extracted passport information
Final Review UI	Allows the user to verify extracted information before submission

1. Face Detection

cv2

- OpenCV library.
- Used for image processing and visualization.
- In this code, it is mainly used for:
 - Converting BGR → RGB.
 - Drawing bounding boxes.
 - Drawing landmark points.
 - Adding text to images.

mediapipe

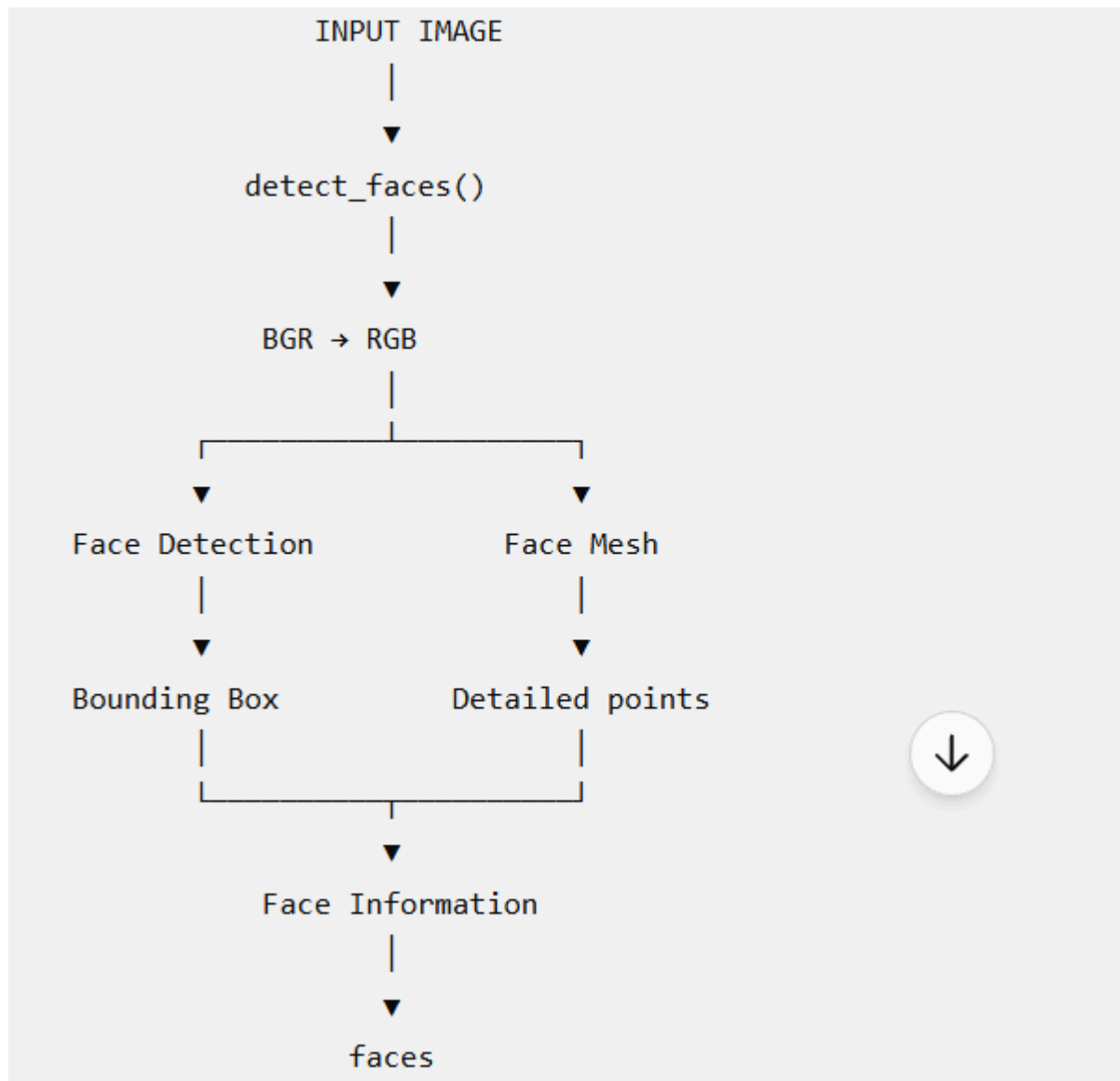
- Google's MediaPipe framework.
- Used here for:
 - Face detection
 - Face landmark detection
 - Face mesh generation

In your project, its responsibility is essentially:

Take an image → detect faces → identify important facial points → generate face mesh points → return the information needed for visa-photo validation.

For example, the application can use these points to determine whether:

- a face is present,
- the face is properly positioned,
- eyes are visible,
- mouth position is appropriate,
- the face is centered, etc.



Component	Purpose
MediaPipe Face Detection	Finds whether a face exists and where it is
Bounding Box	Defines the rectangular area containing the face

Basic Landmarks Gives positions of eyes, nose, mouth and ears

MediaPipe Face Mesh Gives detailed facial geometry

Confidence Score Determines whether detection is reliable enough

OpenCV Handles image conversion and visualization

detect_faces() Performs the actual face analysis

draw_faces() Visually displays the detection results for testing

2. Glasses Detector

After finding the face, it assumes the glasses will be around the **upper-middle portion of the face** using dimensions and Crop the eye region.

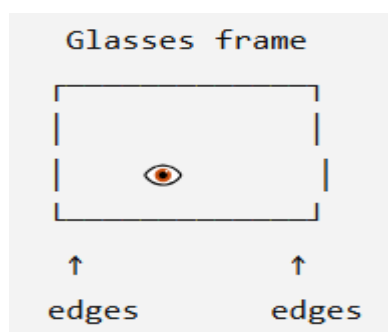
What is an edge?

An edge is basically a location where there is a significant change in brightness/intensity. Sobel and Canny are the methods used to detect these.

Black area White area



↑ (EDGE HERE)

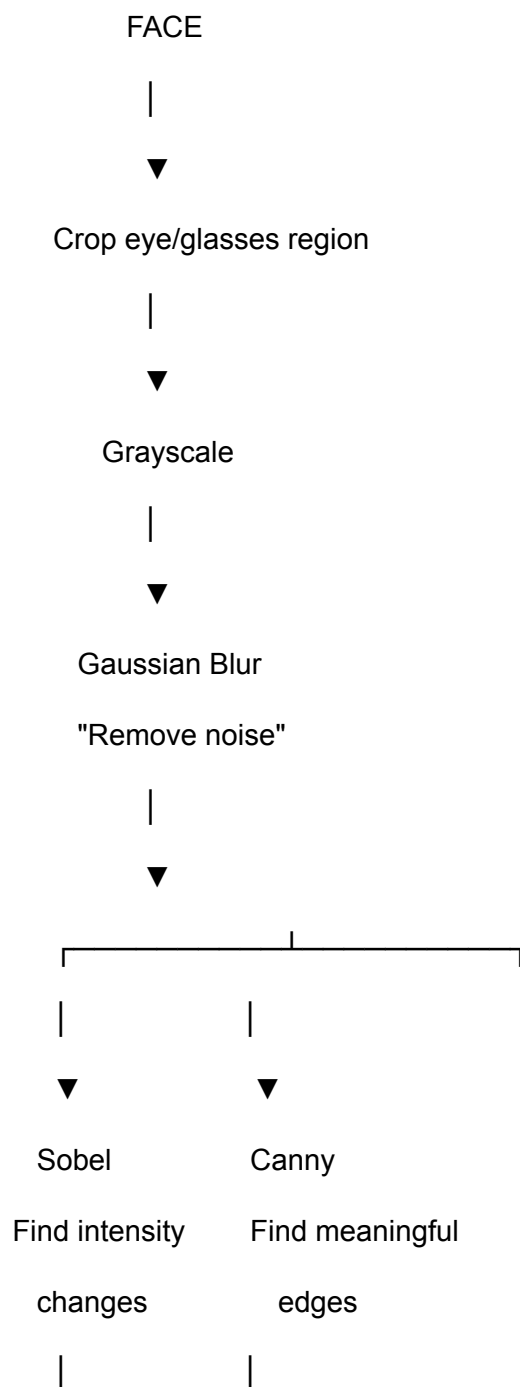


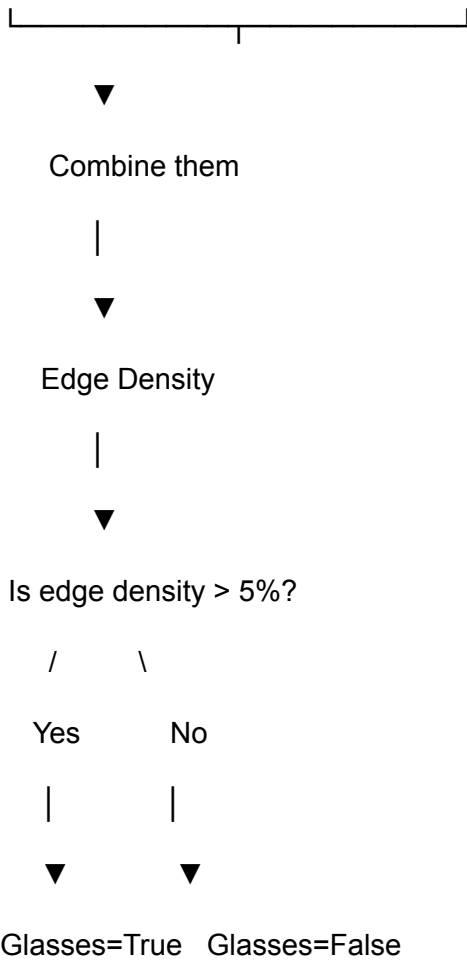
Gaussian blur = slightly smooth the image to reduce noise and tiny unwanted details.

There may be lots of tiny variations caused by:

- skin texture
- camera noise
- individual hairs
- tiny lighting changes
- small wrinkles

These can create unwanted edges. Gaussian blur smooths those small variations

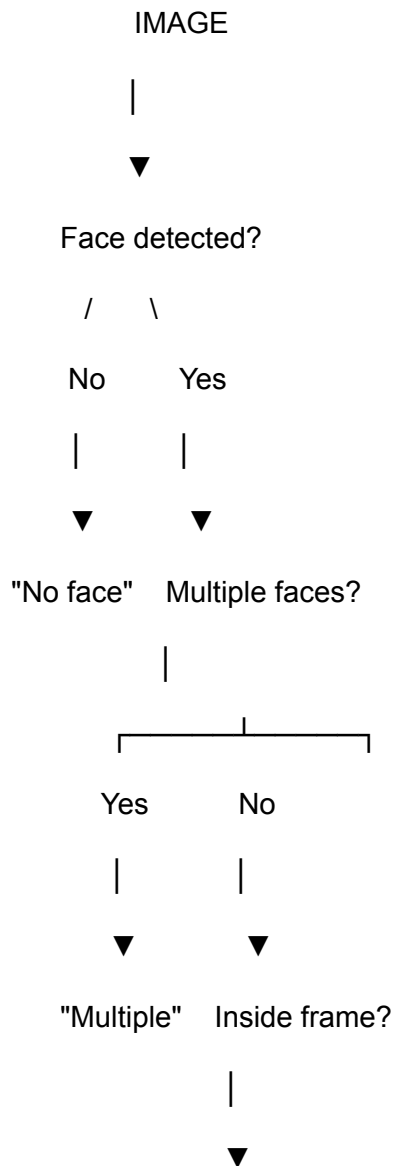




Technique	Simple meaning	Why used here
Eye-region cropping	Keep only the area where glasses are expected	Remove irrelevant parts of the face
Gaussian Blur	Smooth the image and reduce small noise	Prevent tiny unwanted details from creating excessive edges
Sobel	Detect intensity/gradient changes	Highlight boundaries that may belong to the glasses frame

Canny	Detect relatively clean/meaningful edges	Get another representation of the glasses/eye boundaries
Combine	Mix Sobel + Canny results	Use information from both
Edge Density	Measure how many strong-edge pixels exist	Use it as the heuristic signal for glasses detection

3. Image Validator



Face size correct?

/ \

Too small

Too large

|

|



"Come closer"

"Face too close"

|



Face covered?

|



Mouth open? ("Please close your mouth")

|



Eyes closed? ("Eyes are closed")

|



Face not straight? ("Face is not straight")

|



Glasses? ("Please remove your glasses")

|



Correct position?

|



"Great! Look at camera"

4. Passport Data Extraction

Tools used- AWS Textract, AWS Bedrock Anthropic Claude 3 Haiku, AWS SSM Parameter Store, FastAPI Web framework

"""

Passport Document Classification and Extraction Prompt

Defines a detailed prompt for an LLM to classify OCR text from passports into "Front Page," "Back Page," or "Both Pages" using strict MRZ and field presence rules.

Instructs precise extraction of 18 specified fields with confidence scores, outputting a strictly formatted JSON object without inference or assumption.

"""

def construct_claude_prompt() -> str:

return ("""

You are an expert document parser and passport classifier.

The user will provide raw OCR text extracted from a passport or document.

""Document Classification Rules (CRITICAL):

- If the OCR text contains any of the front page fields: Given Name, Surname, Passport Number, Nationality, Country Code, Sex, or Date of Issue, AND the OCR text contains a valid Machine Readable Zone (MRZ) line, classify the document as "Front Page."

- If the OCR text contains any of the back page fields: Father Name, Mother Name, Address, Spouse, file number AND the OCR text does NOT contain a valid MRZ line, classify the document as "Back Page."

- If the OCR text contains at least one front page field AND at least one back page field AND contains a valid MRZ line, classify the document as "Both Pages."

Additional rules:

- The MRZ line must be recognized as a line of two rows of 44 characters each (typical for passports), containing encoded data such as document type, document number, holder's name, nationality, birth date, sex, and expiration date.

- Only consider the MRZ line as valid if it matches expected MRZ format standards.
- Strictly exclude passports numbers or file numbers with special characters or slashes (e.g., "4575/01").

Your classification output must be one of: "Front Page," "Back Page," or "Both Pages" based on the above rules.

Do not classify any document as "Front Page" or "Both Pages" without the valid presence of the MRZ line.

A document should be classified as Both Pages only if it contains a valid MRZ line (Front Page), the '80%' of Front Page fields, and at least '90%' of the Back Page fields.

Your task is to organize this text into a well-structured JSON object. Do not assume or infer any information; only include what is explicitly observed in the input.

CRUCIAL EXTRACTION RULES:

Document Type:

- Extract the document type (single character, typically 'P' for passport)

Personal ID Number:

- Extract ONLY if it matches an 8-10 digit numeric/alphanumeric pattern
- Exclude MRZ line data (e.g., P123456AA0CAN9008010F3301144)
- Ignore values shorter than 8 characters or random sequences

Passport Number and File Number:

- Extract with complete accuracy
- Must be alphanumeric without special characters (e.g., "AF521952")
- Do NOT treat numbers with slashes or special chars (e.g., "4575/01") as valid passport/file numbers

Name Fields (CRITICAL - DO NOT VIOLATE):

- NEVER use "Father Name" or "Mother Name" as the passport holder's "Given Name" or "Surname"

- Parent names are completely separate fields
- If a field contains "Father" or "Mother", it is ONLY a parental field
- Even if parent's name looks like a full name (e.g., "RAJA SRINIVASAN"), NEVER populate Given Name or Surname
- Passport holder's name ONLY comes from fields labeled: "Name", "Given Name", "Surname", "Holder Name"

Address:

- Capture the complete address without truncation or reformatting

Date Format:

- Convert all dates to DD/MM/YY format with slashes
- No underlining or additional formatting

Nationality:

- If multiple nationalities separated by / or , → extract only the first one
- Example: "DANSK/DANISH/DANOISE" → extract "DANSK"

Field Mapping:

- Map extracted text exactly to provided field names
- Do not infer, guess, or assume missing information
- Use empty strings ("") for any field not found or unclear

CONFIDENCE SCORING:

- Assign scores from 0.0 to 1.0 based on OCR clarity
- 1.0 = clear, accurate text
- 0.5 = ambiguous/incomplete
- 0.0 = missing or unreadable

- Include <field>_confidence for each field

OUTPUT FORMAT (STRICT JSON):

- Valid JSON object only, no extra text or commentary
- Include "documentType" field with value: "Front Page", "Back Page", or "Both Pages"
- Include ALL specified fields exactly in this order:

Type, Country Code, Passport Number, Nationality, Surname, Given Name,

Place Of Birth, Place of Issue, Date of Birth, Date of Issue,

Date of Expiry, Sex, Father Name, Mother Name, Spouse Name,

Address, File Number, Personal ID Number

- Use empty strings (") and confidence score 0.0 for missing/unclear fields
- No trailing commas or extra keys

Structure:

```
{
  "documentType": "<Front Page|Back Page|Both Pages>",
  "<field_name>": {
    "value": "<extracted_value>",
    "confidenceScore": <float_score>
  },
  <field_name>": {
    "value": "<extracted_value>",
    "confidenceScore": <float_score>
  }all 18 fields
  ...
}
```

Libraries/ parameters Understanding

a) MIME type checking

The passport `file` type uploaded by the user(e.g., `image/jpeg`) is compared against an expected mapping in `ALLOWED_MIME_TYPES` in `code`

```
ALLOWED_MIME_TYPES = {
```

```
    "jpg": "image/jpeg",  
    "jpeg": "image/jpeg",  
    "png": "image/png",  
    "pdf": "application/pdf"}
```

b) Magic bytes

`python-magic-bin` is a specialized Python package that provides the underlying binary libraries (`libmagic`) needed to **detect file types by looking at their contents**, rather than their file extensions.

Why: even if the extension and header both say "png," (for the uploaded file which is checked by mime) if the actual bytes (actual data inside the file) don't match a PNG signature, it's rejected. This is the real security check; MIME/extension are just first-pass filters.

Every standard file format has a unique **binary signature** (often called **magic bytes** or a **magic number**) baked directly into the very first few bytes of the file.

- **The Extension is just a label:** Changing a filename from `virus.exe` to `photo.jpg` is like taking a bottle of poison and slapping a "Milk" label on it. The outside changes, but the liquid inside is still poison.
- **The Magic Bytes are the DNA:** The operating system and deep-inspection libraries (like `libmagic`) completely ignore the label. They pop open the file, look at the first few bytes of raw data, and know instantly what it is.

c) Pillow

`python-magic` only looks at the magic bytes at the very beginning of the file. Once it sees a valid header, it stops looking. Attackers exploit this blind spot therefore Pillow reads the entire structure, tries to process it, detects the illegal trailing code, and says:

"Corrupted/Malicious!"

Pillow can absolutely be used for validation, and it provides a vital second layer of security and integrity that python-magic cannot provide on its own. While python-magic checks if the beginning of the file looks like an image, Pillow checks if the entire image is actually valid and not corrupted or carrying hidden malicious code.

d) PyMuPDF

PDF to Image Conversion (Rasterization): Converts PDF pages into images (PNG/JPEG)

e) ThreadPoolExecutor

Used in `perform_ocr_on_file()` to OCR multiple PDF pages parallelly— e.g., a 6-page passport PDF (front + back + extra pages) submits 6 OCR calls to Textract in parallel (`max_page_workers=6`) instead of one at a time sequentially.

f) tenacity (retry library)

Tenacity is an advanced, rate-limiting-aware retry library for Python. It is commonly used to wrap network requests to make code resilient against temporary failures.

When working with **AWS** via `boto3`, network drops and API throttling are common. Combining Tenacity with `boto3` exceptions (`BotoCoreError`, `ClientError`, `EndpointConnectionError`) ensures your cloud application automatically recovers from temporary glitches.

You must be careful **not** to retry errors that are permanent (like "Access Denied" or "Resource Not Found"). Retrying those will just waste time and spam the API.

- **EndpointConnectionError (Boto3):**  **Always Retry.** This happens when your local machine completely loses internet connection or AWS DNS fails to resolve.
- **BotoCoreError (Boto3):**  **Conditional.** This is the base exception for low-level client errors (like timeout issues).
- **ClientError (Botocore):**  **Conditional.** This is triggered by AWS service responses. You should **only** retry if the error code indicates a temporary issue (like `ThrottlingException` or `RequestLimitExceeded`).

g) confidence score

Where it comes from: The LLM (Claude 3 Haiku via Bedrock) doesn't just return field values — for each field, it returns a value *and* its own confidence score for that value (presumably self-assessed extraction certainty, e.g., "I'm 92% sure this is the passport number"). This is expected to arrive in a structure like:

```
{
```

```
"documentType": "Front Page",

"passportNumber": {"value": "N1234567", "confidenceScore": 0.95},

"surname": {"value": "SHARMA", "confidenceScore": 0.98},

"givenName": {"value": "RAVI", "confidenceScore": 0.97},

"dateOfBirth": {"value": "1990-05-12", "confidenceScore": 0.80},

}
```

Why field groups exist: A passport front page has a different set of fields than the back page (front: passport number, nationality, DOB, etc.; back: father's name, address, spouse name, etc.). `calculate_average_confidence()` uses `document_type` ("Front Page", "Back Page", or "Both Pages") to know **which fields are actually expected**, so it doesn't penalize a front-page scan for "missing" a back-page-only field like `fatherName`.

Calculation steps (`calculate_average_confidence`):

1. Look up the relevant field list for the given `document_type`.
2. Normalize field names (lowercase, strip non-alphanumerics) so "Passport Number", "passportNumber", "passport_number" all match the same canonical key — this guards against inconsistent LLM output formatting.
3. For each expected field, pull its `confidenceScore` if present and numeric; skip/log fields that are missing or malformed.
4. Average all collected confidence scores, round to 3 decimals.
5. If **zero** valid scores were found, raise an error (rather than silently returning 0.0) — an explicit signal that extraction failed.

Step by Step Summary of this passport data extractor

Pipeline flow:

1. Upload & validate — files are checked for extension, MIME type, magic bytes, size limits, and images are sanitized/re-encoded (via Pillow) to strip metadata and block malicious payloads.
2. OCR (AWS Textract) — PDFs are rendered to PNG per page (via PyMuPDF/`fitz`) and OCR'd; multi-page PDFs are processed concurrently with a `ThreadPoolExecutor`. Images are OCR'd directly in memory. Textract calls have retry/backoff logic (via `tenacity`).

3. Structured extraction (Bedrock/Claude 3 Haiku) — raw OCR text is sent to an LLM to extract structured passport fields, each annotated with a confidence score.
4. Confidence scoring — averages confidence scores across a field group (Front Page / Back Page / Both Pages) to produce an overall document confidence score.
5. API layer — FastAPI exposes `/extract-passport-data` (rate-limited, async, timeout-guarded) and `/healthz` for readiness checks (Textract, Bedrock, Elasticsearch, APM). Errors are logged in detail internally but sanitized before returning to users (no AWS/internal error leakage).
6. Config — environment-specific settings (model ID, Elastic APM config) are pulled from AWS SSM Parameter Store; static settings (timeouts, upload limits, MIME whitelist) are hardcoded in a settings module.

What supports scalability:

a) **Async I/O throughout** — FastAPI endpoints are `async def`, and blocking calls (Textract, Bedrock, file validation) are pushed into a thread executor via `run_blocking_with_timeout`, so the event loop isn't blocked waiting on slow external APIs. This lets one process handle many concurrent requests.

Without this, if Textract takes 3 seconds to respond, the entire process would be frozen for those 3 seconds — unable to accept or process any other request.

`run_in_executor` hands the blocking call to a background thread, freeing the event loop to keep accepting new requests.

Example: User A uploads a passport, and their file is mid-OCR (Textract call in progress, taking 2s). While that thread waits, User B's request arrives at the same time — the event loop is free to accept it, validate it, and kick off User B's OCR in another thread, rather than making User B wait in a queue for User A to finish. One process can now juggle dozens of in-flight requests instead of one at a time.

b) **Bounded concurrency** — `asyncio.Semaphore(max_workers)` in `ocr_llm_extraction_pipeline` and `ThreadPoolExecutor(max_workers=max_page_workers)` in the Textract module cap how many OCR calls run in parallel per request, avoiding resource exhaustion or Textract API throttling.

```
MAX_WORKERS = 3
```

```
semaphore = asyncio.Semaphore(max_workers)
```

Why this matters for scale: Without a cap, if a client uploads 5 files (the max allowed) and each is a 20-page PDF, that's potentially 100 simultaneous Textract calls from *one request*. Multiply by many concurrent users and you'd exhaust memory/threads or get throttled by AWS instantly. The semaphore limits it to 3 files processed concurrently per request, and 6 pages concurrently per file — controlled, predictable resource usage that scales gracefully rather than spiking unpredictably.

Example: 5 files uploaded → semaphore only lets 3 run their OCR pipeline at once; the other 2 wait until a slot frees up. This keeps memory/thread usage bounded and predictable regardless of how many files or how much traffic hits the service.

Step	What happens?	Active Load on Server
Step 1	5 files hit the server. The Semaphore stops 2 of them. Only 3 files start.	3 active files
Step 2	Each of those 3 files is broken into 20 pages. The ThreadPoolExecutor grabs them but says: <i>"I will only process 6 pages at a time."</i>	18 active pages (3 × 6) instead of 100!
Step 3	Pages finish one by one. Slots clear up. Eventually, a whole file finishes, freeing a Semaphore ticket for the files waiting outside.	Smooth, steady traffic.

c) Rate limiting (slowapi)

Protects the service itself from being overwhelmed by a single client.

```
limiter = Limiter(key_func=get_remote_address,
default_limits=[f'{RATE_LIMIT_REQUESTS}/{RATE_LIMIT_PERIOD}'])
app.add_middleware(SlowAPIMiddleware) app.state.limiter = limiter
@app.post("/extract-passport-data")
@limiter.limit(f'{RATE_LIMIT_REQUESTS}/{RATE_LIMIT_PERIOD}') async def
process_passport_images(...):
```

Settings: `RATE_LIMIT_REQUESTS = 30`, `RATE_LIMIT_PERIOD = "minute"` — so 30 requests/minute per client IP (`get_remote_address`).

Why this matters for scale: Scalability isn't just "handle more load" — it's also "don't let one bad actor or buggy client (eg- The 5th passport image is corrupted. Instead of skipping it, the script gets confused and re-uploads that same corrupted passport **200 times in a row** without pausing.) consume all capacity, starving everyone else." Rate limiting ensures fair resource allocation across many clients,

which is what lets the service scale to serve *many users* rather than being monopolized by one.

Example: A client script with a bug in a retry loop tries to hit `/extract-passport-data` 200 times in a minute. After 30, they get:

request, 429, "Rate limit exceeded. Please slow down your requests."

d) Upload/file size guards

```
MAX_FILE_SIZE_BYTES = int(MAX_FILE_SIZE_MB * 1024 * 1024) # 10 MB per file
MAX_TOTAL_UPLOAD_BYTES = int(MAX_TOTAL_UPLOAD_MB * 1024 * 1024) #
40 MB per request
MAX_UPLOAD_FILES = 5
```

Why this matters for scale: Every request consumes memory proportional to file size (since files are read fully into memory, not streamed to disk for images). Without a cap, one client could upload a 2GB "image" and consume all available server memory, potentially crashing the process and affecting every other concurrent user. Bounding size per-file and per-request keeps memory usage per-request predictable, which is essential for capacity planning (e.g., "this instance can safely handle N concurrent requests given M memory").

Example: Instance has 2GB RAM. If file size were unbounded and 20 concurrent users each uploaded a 500MB file, that's 10GB needed — instant crash. With `MAX_FILE_SIZE_MB=10` and `MAX_TOTAL_UPLOAD_MB=40`, worst case for 20 concurrent requests is 800MB — safely within capacity.

What is "Resilience" and how is it handled?

Resilience = the system's ability to **survive partial failures gracefully** — transient errors, slow dependencies, or overload — without crashing, hanging, or corrupting results.

a) Retries with exponential backoff (**tenacity**)

`_detect_document_text()` (Textract call) is wrapped with `@retry(...)`: up to `GLOBAL_MAX_RETRIES` (3) attempts, exponential backoff (`wait_exponential`, starting at 0.5s, capped at 8s), and only retries on transient AWS errors (`BotoCoreError`, `ClientError`, `EndpointConnectionError`) — not on non-retryable errors.

Each retry is logged (`_log_retry`) and counted (`OCR_RETRY_COUNTER`), so transient blips are self-healed without manual intervention, and are still observable.

b) Timeouts on every blocking/external call

OCR_TASK_TIMEOUT = 65

LLM_TASK_TIMEOUT = BEDROCK_TIMEOUT + 5

file validation given 10.0s

`run_blocking_with_timeout()` wraps OCR, LLM, and file-validation calls with `asyncio.wait_for`, using dedicated timeout constants (`OCR_TASK_TIMEOUT`, `LLM_TASK_TIMEOUT`, a 10s validation timeout). This prevents one slow/hung dependency from tying up a request (or the whole event loop) indefinitely — it fails fast instead.

A dedicated `PipelineTimeoutError` distinguishes "timed out" from other failures, letting the API return a specific `504` rather than a generic `500`.

What it solves: Without timeouts, a hung dependency (e.g., Textract stops responding but doesn't return an error) would hold that request's thread/resources **forever**. Over time, more and more requests pile up waiting, threads get exhausted, and the whole service becomes unresponsive — a classic cascading failure.

Example: Bedrock's LLM call hangs due to a network partition — no response, no error, just silence.

- Without a timeout: that thread is stuck indefinitely; if this happens repeatedly, all worker threads eventually get stuck the same way, and the service can no longer process *any* request.
- With the timeout: after 65s, `asyncio.TimeoutError` fires → `PipelineTimeoutError` is raised → the route handler catches it and returns a clean `504`

The thread is freed, resources are reclaimed, and the service keeps serving other requests normally.

c) Rate limiting (`slowapi`)

- Protects the service from being overwhelmed by request bursts (`RATE_LIMIT_REQUESTS` per `RATE_LIMIT_PERIOD`), with a dedicated `429` handler — a form of self-protection/resilience against overload.

d) Bounded concurrency

- `Semaphore/ThreadPoolExecutor` caps limit how many OCR/LLM operations run simultaneously, preventing resource exhaustion (memory, threads, downstream throttling) under high load.

e) Input validation & sanitization as a resilience layer

Extension/MIME/magic-byte checks and image re-encoding stop malformed or malicious files from ever reaching OCR/LLM stages, where they'd otherwise cause unpredictable failures downstream.

What it solves: Malformed, corrupted, or malicious files can cause **unpredictable failures deep inside the pipeline** — e.g., PyMuPDF crashing on a corrupt PDF, or Textract throwing an obscure error on a malformed image, or (in the worst case) a crafted file exploiting a parsing library vulnerability. Catching this *before* it reaches OCR/LLM stages means failures happen early, cheaply, and predictably (a clean `400` response) instead of late, expensively, and unpredictably (a crash or hang mid-pipeline).

Example: Someone uploads a file named `passport.png` that's actually a corrupted/truncated binary. Magic byte + Pillow validation catches this immediately.

5. Visa Chatbot

This is a conversational AI chatbot for visa information that converts natural-language user questions into MongoDB queries, executes them, and returns human-friendly, formatted answers. It's built as a multi-step agentic workflow using LangGraph.

Tools- LangGraph, LangChain, Google Gemini, MongoDB

What it does (end-to-end flow)

1. User asks a question in plain English (e.g., "What documents do I need for India?", "Show me tourist visa costs for USA", "What's the age limit for a child visa in Albania?").
2. Router node — classifies the query into a category (`document_query`, `age_query`, `cost_query`, `visa_options_query`, `visa_mode_query`, or `general_query`) using simple keyword matching.
3. Text-to-MQL node — sends the user's query, the MongoDB schema, and conversation history to an LLM, which generates a structured MongoDB aggregation pipeline (JSON) as a query plan.
4. Query execution — the generated pipeline is run directly against MongoDB (`collection.aggregate(pipeline)`).
5. Format-response node — the raw MongoDB results are sent back to the LLM with formatting instructions (convert codes to readable text, add emojis, structure cost breakdowns, etc.) to produce a polished, user-facing reply.
6. Conversation memory — each session (`thread_id`) is checkpointed so follow-up questions retain context.

Domain / Data model

The uploaded image lists the actual MongoDB collections backing this (a visa-services database): `VisaCountries`, `VisaEntryType`, `VisaFeeType`, `VisaMode`, `VisaProcessingOption`, `VisaPurpose`, `VisaSKU`, `VisaStayDuration`, `VisaStayWindow`, `VisaValidity`, `Timezone`. The code's `DATABASE_SCHEMA` docstring describes six of these in detail (`Country`, `CountryAgeRules`, `CountryDocument`, `VisaSKU`, `SkuPricing`, `VisaMode`) — essentially a reference/lookup system for visa rules, documents, pricing, and processing options across countries, encoded with short domain codes (e.g., `TOURS`=Tourist, `EVIS`=eVisa, `ST30D`=30-day stay) that the LLM is taught to translate to/from natural language.

Text to MQL

Is "Text-to-MQL" a library or MongoDB-developed tool?

No — it's not a library, product, or official MongoDB tool. MQL stands for MongoDB Query Language (the JSON-based query/aggregation syntax MongoDB uses — things like `$match`, `$lookup`, `$group`, etc.). "Text-to-MQL" is simply a descriptive term/pattern name for the general technique of using an LLM to translate a natural-language question into an MQL query — analogous to the much more commonly known term "Text-to-SQL."

MongoDB does have some official/semi-official AI tooling in this space (e.g., natural-language query assistants in MongoDB Atlas, MongoDB's Compass "AI query" features, and various MongoDB-published sample apps/tutorials on LLM-to-MQL generation), but there's no single canonical "Text-to-MQL library" you `pip install`. In the project you showed me, it's just a hand-written function (`text_to_mql()`) that constructs a prompt and calls whatever LLM is configured — it's a custom implementation of the pattern, not a package.

How Text-to-(SQL/MQL) generally works, conceptually

This is a well-established pattern in applied LLM engineering, used for any structured query language (SQL, MQL, GraphQL, SPARQL, etc.). The core idea: an LLM translates human language into a formal query, which is then executed by a real database engine — the LLM never touches the actual data directly.

Typical pipeline:

1. Schema/context injection — The system prompt includes a description of the database structure: table/collection names, field names, data types, relationships between tables, and often sample values or enum meanings (e.g., "status can be ACTIVE or INACTIVE"). The LLM has no built-in knowledge of *your* specific database — it must be told what exists.
2. Few-shot examples — Most implementations include several example (question → correct query) pairs directly in the prompt. This dramatically improves reliability

because it shows the model the exact query dialect, syntax conventions, and structure to follow, rather than relying purely on the model's general training knowledge of the query language.

3. Natural language question — The user's actual question is appended.
4. LLM generates a structured query — The model outputs what it believes is the correct query (SQL string, MongoDB pipeline as JSON, etc.), ideally constrained to *only* output the query (often enforced by prompting "return only valid JSON/SQL, no explanation").
5. Parsing/validation — The raw LLM output is cleaned up (stripping markdown fences, extracting the JSON/SQL substring, validating it parses correctly) since LLMs don't always follow formatting instructions perfectly.
6. Query execution — The parsed query is run against the *actual* database using a normal driver/client (e.g., `pymongo`, `psycopg2`). This is a critical separation: the LLM only produces the query text; a conventional, deterministic database engine executes it and returns real data.
7. (Optional) Result formatting — Raw query results (often verbose/technical) are sometimes passed back to an LLM a second time to be turned into a readable, natural-language answer for the end user — this is the pattern the project used (`format_results_for_user`).

Why this approach is used:

- Lets non-technical users query a database in plain English without knowing SQL/MQL syntax.
- Powers "chat with your data" style products and internal BI tools.

Known weaknesses (industry-wide, not project-specific):

- LLMs can generate syntactically valid but *logically wrong* queries (misinterpreting intent, wrong joins, wrong filters).
- Schema complexity (many tables/collections, ambiguous naming) degrades accuracy significantly.
- Security risk if not carefully sandboxed — a poorly constrained implementation could allow destructive operations (`$out`, `drop`, `delete`) if the LLM's output isn't restricted to read-only operations.
- Requires either a strong general-purpose LLM or heavy prompt engineering (many examples, strict formatting rules) to get reliable structured output — which is exactly why the project you shared leaned so heavily on detailed prompts and defensive JSON parsing.

COTALITY- CORE LOGIC

- Designed, developed and deployed end-to-end data pipelines on Google Cloud Platform (GCS, BigQuery) orchestrated through Apache Airflow, onboarding datasets from staging into governed EDR and historical layers.
- Built and configured Airflow DAGs and their constituent tasks — including scheduling, task versioning, retries, dependency configuration and failure alerting — and promoted workflows across DEV, INT and PRD environments.
- Owned end-to-end onboarding of Model Input and Model Output tables across environments, covering dataset creation, table provisioning and schema setup in line with team-approved definitions.
- Implemented and maintained BigQuery external tables over GCS (wildcard path patterns, header handling, file-format configuration) to ensure newly delivered reference files became query-visible without table recreation.
- Developed BQ-to-BQ ingestion flows moving model-generated output from source staging locations through staging, EDR target and history layers on a monthly cadence, ensuring reliable operationalization of model results.
- Applied ETL standardization logic to align field names, data types and schema versions to the agreed data contract, maintaining consistency across DEV, INT and PRD.
- Enforced data quality controls including deduplication, null and type checks, business-key handling and primary-key validation, with uniqueness verification to guarantee stable downstream behaviour.
- Applied data governance standards — table and column descriptions, IAM/AD-based access controls, retention and history rules — and placed data in approved EDR/HIST layers.
- Developed and optimized SQL queries and stored procedures in BigQuery for data loading, transformation, archival and historical dataset migration.
- Monitored workflow executions in Airflow and performed validation using BigQuery and the gcloud CLI; investigated, debugged and resolved pipeline failures, permission issues and configuration defects.
- Conducted cross-environment configuration comparison and compatibility testing during platform upgrades (SDK, Python and Airflow version upgrades) to safeguard pipeline stability.
- Managed access provisioning and credential verification for pipeline and model-related components in coordination with platform and security teams.

- Produced review-ready validation evidence for stakeholders and partnered with QA teams to explain requirements, support test cycles and resolve defects.

- Authored and maintained technical documentation in Confluence covering pipeline architecture, DAG-level implementation and operational runbooks; tracked deliverables through Jira with regular status reporting.

Technical Skills: GCP (GCS, BigQuery) · Apache Airflow · gcloud CLI · SQL & Stored Procedures · Python · ETL/ELT · Data Modeling · Data Quality & Validation · Data Governance, Lineage & Retention · Confluence · Jira

Tools- Oracle, GCP(GCS, BigQuery), Airflow, gcloud

Model Input

The reference data fed into the model.

Here: the manually verified CSV of entity-parent relationships.

Purpose: provide trusted examples (ground truth) the model uses to resolve ownership.

Model Output

The results produced by the model after processing transaction/entity data.

Here: resolved legal-entity matches with fields like entity_id, matched_name, ultimate_parent, confidence_score.

Purpose: provide usable downstream data for EDR tables, reporting, and portfolio monitoring.

Simple way to remember:

Input = what goes into the model

Output = what comes out of the model

Why ETL is still required in model output:

1. Operationalization

Model writes to a source location (often temp/staging).

ETL moves it to governed target datasets used by business systems.

2. Standardization

Rename/align fields to required contract.

Apply schema/version consistency across DEV/INT/PRD.

3. Data quality

Deduplicate, type checks, null checks, PK logic on entity_id.

Ensure stable downstream behavior.

4. Governance and security

Apply table/column descriptions, AD/IAM access, retention/history rules.

Put data in approved EDR/HIST layers.

5. Scheduling and reliability

Run monthly cadence, retries, monitoring, failure alerts.

So:

Model = generates intelligence.

ETL = makes that intelligence reliable, governed, and consumable in production.

Roles & Responsibilities:

1. Own end-to-end onboarding of Model Input and Model Output tables across environments (DEV first, then INT/PRD), including correct dataset, table, and schema setup.

2. Manage Model Input external-table behavior in GCS (entity_parent_reference path, wildcard pattern, header handling), and ensure new matching files are query-visible without table recreation.

3. Enforce data quality expectations and keys:

Model Input: treat entity_name as business key and handle duplicate risk from wildcard files.

Model Output: set/validate entity_id as PK metadata and verify uniqueness with checks.

Maintain schema and metadata alignment with latest team-approved definitions (especially the final 10-column Model Output schema and column descriptions).

4. Operate and troubleshoot execution via PMTFYI/CLI reliably (project context, permissions, SQL/JSON escaping, multi-statement limitations), and provide review-ready validation evidence to stakeholders.

Understanding:

Based on my understanding of requirements document, I have mentioned the steps below to be implemented. Please let me know if corrections are required in these steps:

1) Model Input — Ultimate Owner

Create edr_ent_property_consumer_pin.ext_entity_parent_reference table in REG and connect it to legal_entity_resolution_manual_verified_parents.csv

2) Model Output:

- Create staging table: stg_business_corporate_hierarchy (in clgx-idap-bigquery-reg-383c)

- Create EDR target table: edr_ent_business_corporate_hierarchy.entity_owner_link (in reg)

- Create history table: edr_ent_business_corporate_hierarchy_hist.entity_owner_link (in reg)

- Build the monthly BQ-to-BQ copy DAG:

Source: clgx-idap-bigquery-int-278e.stg_default_dataset.legal_entity_matches_final

- dataflow:

INT.stg_default_dataset.legal_entity_matches_final >

REG.stg_business_corporate_hierarchy >

REG.edr_ent_business_corporate_hierarchy.entity_owner_link >

REG.edr_ent_business_corporate_hierarchy_hist

